



UNIVERSIDADE DO MINHO
ESCOLA DE ENGENHARIA

RELATÓRIO

PROGRAMAÇÃO ORIENTADA AOS OBJETOS

Grupo 5:

91672 LUÍS FERREIRA
93258 BERNARDO LIMA
91637 HENRIQUE FARIA

Professor:

António Nestor Ribeiro

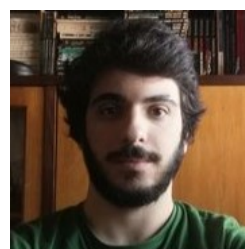
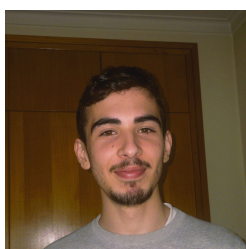




Tabela de conteúdos

1	Introdução	2
2	Arquitetura do Projeto	3
3	Diagrama de Classes	3
4	Classes	4
4.1	<i>Main</i>	4
4.2	<i>View</i>	4
4.3	<i>Controller</i>	4
4.4	Model	4
4.5	Atividade	4
4.6	AtividadeDist	5
4.7	AtividadeDistAlt	5
4.8	AtividadeRep	5
4.9	AtividadeRepPesos	6
4.10	Hard	6
4.11	Utilizador	6
4.12	UtilizadorAmador	6
4.13	UtilizadorOcasional	6
4.14	UtilizadorProfissional	7
4.15	UtilizadorManager	7
4.16	PlanoTreino	7
4.17	SystemDate	7
4.18	Exceptions	7
5	Persistência de Dados	8
6	Funcionalidades do Programa	9
6.1	Gestão de Utilizadores	9
6.2	Gestão de Atividades	9
6.3	Gestão de Planos de Treino	9
6.4	Estatísticas (<i>Queries</i>)	10
6.5	Funcionalidades Adicionais	10
7	Especificações Técnicas das Funcionalidades	11
7.1	Gestão de Utilizadores	11
7.2	Gestão de Atividades Físicas	11
7.3	Gestão de Planos de Treino	11
7.4	Estatísticas (<i>Queries</i>)	11
7.5	Funcionalidades Adicionais	12
8	Conclusão	13



1 Introdução

O projeto que nos foi proposto tem como objetivo desenvolver uma aplicação para a gestão de atividades e planos de treino de praticantes de atividades físicas. A aplicação abrange diversas atividades destinadas a diferentes tipos de atletas onde, dependendo destes dois pontos, será calculado o gasto calórico.

Os utilizadores poderão criar e registar planos de treino personalizados, indicando as atividades a serem realizadas e a sua recorrência. O foco principal da aplicação é permitir que os utilizadores registem a execução das atividades e forneçam indicadores relevantes sobre o seu treino, como calorias consumidas, distância percorrida e elevação acumulada.

Além disso, a aplicação deverá ter a capacidade de gerar planos de treino personalizados com base nos objetivos individuais dos utilizadores, em termos de exercício e gasto calórico esperado.

De forma a completar o projeto foram também criadas algumas *queries* de forma a saber qual foi o utilizador (user) que realizou mais atividades, percorreu a maior distância, mais calorias consumiu (desde sempre ou durante um determinado espaço temporal), qual o tipo de atividade mais realizada na *app* e qual o plano de treino mais exigente tendo em conta as calorias propostas.

Para simular o funcionamento da aplicação em *use cases* reais, também foi implementada uma funcionalidade que permite avançar no tempo.

2 Arquitetura do Projeto

No âmbito da arquitetura do nosso projeto, seguimos o modelo **MVC** (*Model-View-Controller*) e o paradigma de programação orientada a objetos. Neste modelo, o *Model* é responsável pela gestão dos dados e lógica da aplicação, a *View* encarrega-se de apresentar esses dados aos utilizadores, enquanto o *Controller* atua como um intermediário que coordena as interações entre o *Model* e a *View*. A adoção do **MVC** visa separar claramente os diferentes componentes da aplicação, facilitando o desenvolvimento, a manutenção e a escalabilidade do código. Além disso, este padrão promove o encapsulamento e a modularidade das suas partes.

Quanto ao paradigma de programação orientada a objetos, este baseia-se nos conceitos: encapsulamento, herança e polimorfismo. O encapsulamento refere-se a ocultar os detalhes de implementação de uma classe, permitindo o acesso apenas aos métodos e atributos necessários. A herança permite que uma classe herde atributos e métodos de outra, facilitando a reutilização de código e a criação de hierarquias de classes. O polimorfismo permite tratar objetos de classes diferentes uniformemente, utilizando métodos com o mesmo nome, mas comportamentos distintos. Estes conceitos proporcionam uma estrutura sólida e modular para o desenvolvimento de *software*.

Os conceitos citados anteriormente são aspetos fundamentais que são enfatizados nesta Unidade Curricular.

3 Diagrama de Classes

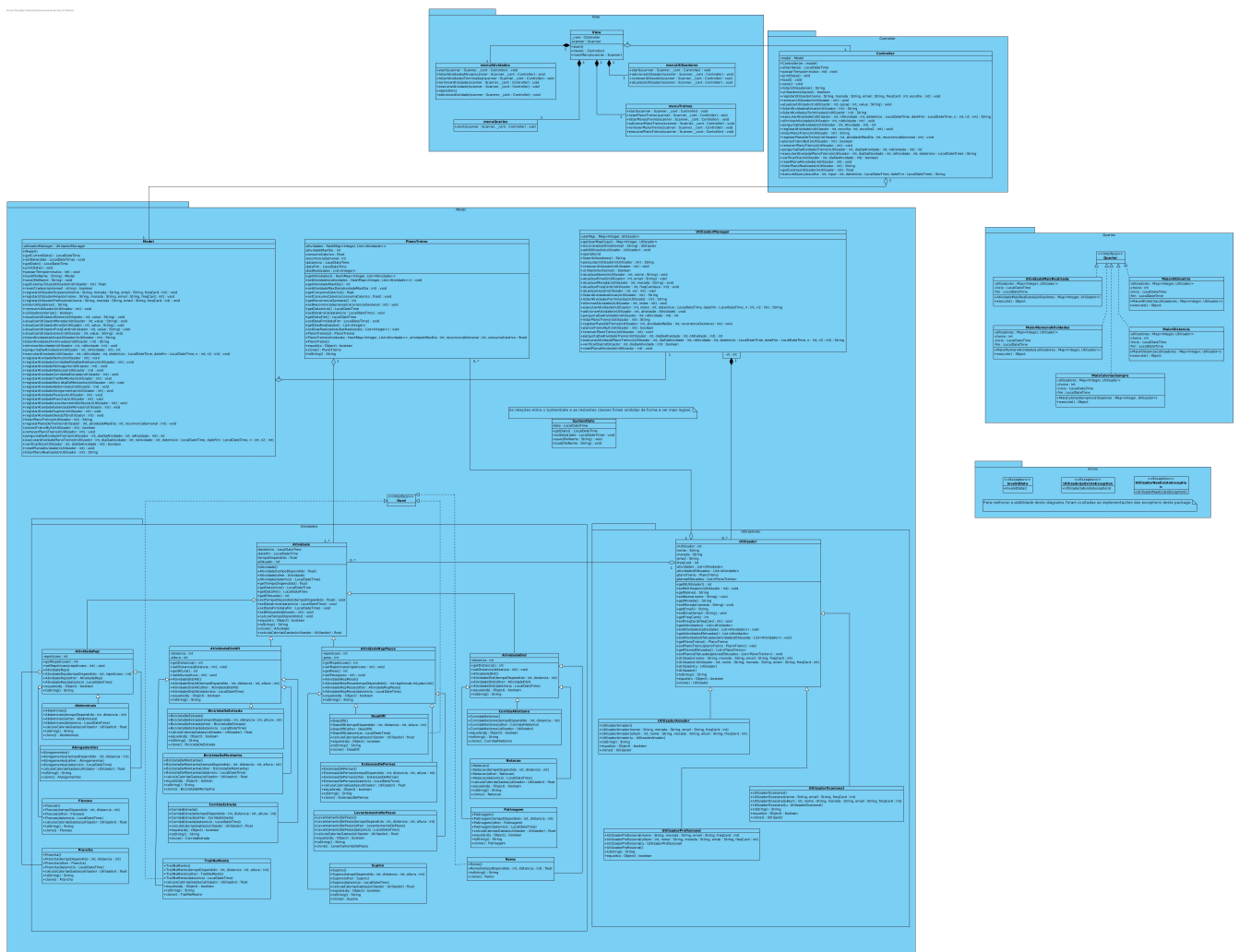


Figure 1: Para ver melhor, imagem e .vpp no github

4 Classes

4.1 *Main*

Responsável pela inicialização do programa. O método `main` inicia o *Controller*, que atua como ligação entre *Model* e *View*;

4.2 *View*

A classe *View* é responsável por interagir com o *user* através do terminal, apresentando menus e recebendo opções selecionadas pelo mesmo. Utiliza um objeto da classe *Controller* para aceder e manipular dados, sendo retornadas as informações ou notificações de sucesso/falha. A classe possui um método `start()` que inicia a interação com o *user*, mostrando o menu principal. O método `mainMenu()` exibe as opções disponíveis para o *user* e, dependendo da escolha feita, redireciona para outros menus específicos, como o menu de utilizadores, de atividades, de plano de treino ou de consultas. Além disso, a classe também possui opções para casos como guardar o estado da aplicação, passar o tempo, ou sair do programa.

4.3 *Controller*

```
private Model m;
```

A classe que atua como intermediária entre *View* e *Model*, tendo a função de chamar os métodos disponíveis do *Model* necessários à *View*, sem aceder diretamente aos dados da aplicação.

4.4 *Model*

```
private UtilizadorManager utilizadorManager;
```

Model é a classe que assegura a parte das regras e camada computacional (a parte lógica/matemática do programa). Um objeto *Model* é construído a partir do *UtilizadorManager*, que possui vários métodos de gestão da aplicação pois, possibilita ao *Model* manipular a informação.

4.5 *Atividade*

```
private LocalDateTime dataInicio;  
private LocalDateTime dataFim;  
private float tempoDespendido;  
int efetuado;
```

Esta classe reflete uma abstração de atividades, representando várias tarefas que podem ser realizadas. Possui atributos como tempo despendido, datas de início e fim e um inteiro que indica se a atividade já foi efetuada ou não. Fornece um método abstrato para calcular as calorias gastas durante a atividade, que deve ser implementado pelas subclasses.

4.6 AtividadeDist

```
private int distancia;
```

A classe **AtividadeDist** é uma subclasse da classe **Atividade**, que representa atividades que envolvem uma distância percorrida, como corrida e remo. Herda os atributos e métodos da classe pai **Atividade** e adiciona um novo atributo **distancia** que regista a distância percorrida durante a atividade. Esta classe fornece métodos para definir e obter a distância. Adicionalmente, declara um método abstrato **calculaCaloriasGastas**, que deve ser implementado pelas subclasses e que calcula as calorias gastas durante a atividade, com base nas características do utilizador fornecido.

Herdam da **AtividadeDist** as seguintes classes:

- Remo
- Natacao
- Patinagem
- CorridaAtlestismo

4.7 AtividadeDistAlt

```
private int distancia;  
private int altura;
```

A classe **AtividadeDistAlt** é uma subclasse da classe **Atividade**, representando atividades que envolvem uma distância percorrida e uma altura alcançada, como *trail* no monte e andar de bicicleta no monte. Herda os atributos e métodos da classe pai **Atividade** e adiciona dois novos atributos chamados **distancia** e **altura** para registar a distância percorrida e a altura alcançada durante a atividade. Esta classe fornece métodos para definir e obter a distância e altura. Além disso, declara um método abstrato **calculaCaloriasGastas**, que deve ser implementado pelas subclasses para calcular as calorias gastas durante a atividade, com base nas características do utilizador fornecido.

Herdam da **AtividadeDistAlt** as seguintes classes:

- TrailNoMonte
- CorridaEstrada
- BicicletaDeEstrada
- BicicletaDeMonte

4.8 AtividadeRep

```
private int repeticoes;
```

Tal como as restantes, a classe **AtividadeRep** é uma extensão da classe **Atividade**, representando atividades que envolvem repetições, como abdominais, flexões e outros. Herda os atributos e métodos da classe pai **Atividade** e adiciona um novo atributo chamado **repeticoes** para registar as repetições feitas durante a atividade. Fornece métodos para definir e obter as repetições. Declara, também, um método abstrato **calculaCaloriasGastas**, que deve ser implementado pelas subclasses para calcular as calorias gastas durante a atividade, com base nas características do utilizador fornecido.

Herdam da **AtividadeRep** as seguintes classes:

- Flexoes
- Abdominais
- Alongamentos
- Prancha

4.9 AtividadeRepPesos

```
private int repeticoes;  
private int peso;
```

Por fim, há a classe `AtividadeRepPesos`, que é uma extensão da classe `Atividade`, representando atividades que envolvem repetições e pesos, como supino e *deadlift*. Herda os atributos e métodos da classe pai `Atividade` e adiciona dois novos atributos chamados `repeticoes` e `peso` para registrar as repetições feitas e o peso utilizado durante a atividade. A classe fornece métodos para definir e obter as repetições e o peso. Também declara um método abstrato `calculaCaloriasGastas`, que deve ser implementado pelas subclasses para calcular as calorias gastas durante a atividade, com base nas características do utilizador fornecido.

Herdam da `AtividadeRepPesos` as seguintes classes:

- `Supino`
- `LevantamentodePesos`
- `ExtensaoDePernas`
- `Deadlift`

4.10 Hard

Interface implementada em atividades, de modo a saber quais são classificadas como sendo difíceis e quais não são.

4.11 Utilizador

```
private static int nUtilizadorCounter = 1;  
private int nUtilizador;  
private String nome;  
private String morada;  
private String email;  
private int freqCard;  
private List<Atividade> atividades;  
private List<Atividade> atividadesEfetuadas;  
private PlanoTreino planoTreino;  
private List<PlanoTreino> planosEfetuados;
```

A classe `Utilizador` é uma classe abstrata que serve como modelo para representar diferentes tipos de utilizadores. Possui atributos como `nome`, `morada`, `email` e frequência cardíaca, além de listas de atividades e planos de treino. A classe também possui métodos para aceder e modificar os atributos, construtores para criar as instâncias da classe e métodos abstratos que devem ser implementados pelas subclasses.

4.12 UtilizadorAmador

A classe `UtilizadorAmador` é uma subclasse de `Utilizador` que representa um tipo específico de utilizador, designado como amador. Herda as características básicas de um utilizador, como `nome`, `morada`, `email` e frequência cardíaca média, e adiciona funcionalidades específicas, como os métodos `toString()`, `equals()` e `clone()` para fornecer uma representação textual, verificar igualdade e permitir a cópia de instâncias do utilizador amador.

4.13 UtilizadorOcasional

A classe `UtilizadorOcasional` é uma subclasse de `Utilizador`, representando um tipo específico de utilizador no sistema, designado como ocasional. Herda as características básicas de um utilizador, como `nome`, `morada`, `email` e frequência cardíaca média, e adiciona funcionalidades específicas, como os métodos `toString()`, `equals()` e `clone()` para fornecer uma representação textual, verificar igualdade e permitir a cópia de instâncias do utilizador ocasional.

4.14 UtilizadorProfissional

A classe `UtilizadorProfissional` é uma subclasse de `Utilizador`, representando um tipo específico de utilizador no sistema, designado como profissional. Herda as características básicas de um utilizador, como nome, morada, email e frequência cardíaca média, e adiciona funcionalidades específicas, como os métodos `toString()`, `equals()` e `clone()` para fornecer uma representação textual, verificar igualdade e permitir a cópia de instâncias do utilizador profissional.

4.15 UtilizadorManager

```
private Map<Integer, Utilizador> userMap = new HashMap<>();
```

A classe `UtilizadorManager` é responsável por gerir os utilizadores. Armazena os utilizadores num mapa, permitindo adicionar, remover, atualizar e pesquisar utilizadores. Além disso, também lida com operações relacionadas a atividades, como adicionar, remover e executar atividades para um determinado utilizador. A classe também é responsável por gerir os planos de treino dos utilizadores, permitindo registar, remover e executar atividades de acordo com um plano predefinido.

4.16 PlanoTreino

```
private HashMap<Integer, List<Atividade>> atividades;  
private int atividadeMaxDia;  
private float consumoCalorico;  
private int ocorrenciaSemanal;  
private LocalDateTime dataInicio;  
private LocalDateTime dataFim;  
private List<Integer> diasRealizados;
```

A classe `PlanoTreino` representa um plano de treino. Armazena informações sobre as atividades planeadas para cada dia da semana, a quantidade máxima de atividades por dia, a recorrência semanal do plano, o consumo calórico estimado, a data de início e fim do plano e os dias em que as atividades foram realizadas. Esta classe tem métodos para aceder e modificar essas informações, além de fornecer métodos para clonar o plano, verificar a igualdade entre planos e gerar uma representação em formato de *string* do plano de treino.

4.17 SystemDate

```
private LocalDateTime;
```

Esta classe permite obter, definir, guardar e carregar a data do sistema num ficheiro, facilitando a persistência da data entre diferentes execuções do programa.

4.18 Exceptions

Para lidar com os casos de exceção, como na tentativa de criação de um *user* já existente, ou na tentativa de acesso não autorizado a um plano de treino, por exemplo. Também cobrimos, outras exceções, focando em datas inválidas. Assim, foram criados 3 tipos de *exceptions*, para lidar com os casos referidos:

```
UtilizadorJaExisteException  
UtilizadorNaoExisteException  
InvalidData
```

5 Persistência de Dados

A serialização em *Java* trata-se do processo de converter um objeto *Java* numa sequência de *bytes*, que pode ser posteriormente armazenado em ficheiro, permitindo que o estado do objeto seja guardado e reutilizado posteriormente.

Primeiramente, para ser possível guardar, é necessário seleccionar as classes que têm a informação completa para representar o estado desejado. Conseguindo posteriormente que os objetos criados sejam serializados. Isto significa que o estado atual dos objetos será convertido numa sequência de *bytes*. Para escolher quando guardar, foi definida, no *Model*, uma função de guardar que será chamada a partir do *Controller* sempre que houver uma mudança entre menus na *View*, na tentativa de guardar qualquer tipo de alteração realizada ao estado do programa.

6 Funcionalidades do Programa

6.1 Gestão de Utilizadores

- Registo de utilizador: O sistema permite o registo de utilizadores, onde é necessário fornecer informações como nome, *email*, morada e frequência cardíaca. Adicionalmente, durante o registo, o utilizador pode especificar o seu tipo: ocasional, amador ou profissional.
- Adicionar Utilizador: Permite adicionar um novo utilizador ao sistema, verificando se o utilizador fornecido é válido.
- Listar Utilizadores: Oferece a funcionalidade de listar todos os utilizadores registados no sistema.
- Pesquisar Utilizador: Permite pesquisar um utilizador com base no seu número de identificação único.
- Remover Utilizador: Permite remover um utilizador do sistema com base no seu número de identificação único.
- Atualizar Informações do Utilizador: Possibilita atualizar as informações de um utilizador, como nome, *email*, morada e frequência cardíaca.

6.2 Gestão de Atividades

- Adicionar Atividade: Permite adicionar uma nova atividade a um utilizador existente no sistema.
- Listar Atividades Ativas: Mostra todas as atividades ativas de um utilizador específico.
- Listar Atividades Terminadas: Exibe todas as atividades terminadas por um utilizador.
- Executar Atividade: Permite que um utilizador execute uma atividade, fornecendo detalhes como a data de início, a data de fim e os parâmetros específicos da atividade, devolvendo as calorias gastas.
- Eliminar Atividade: Oferece a funcionalidade de remover uma atividade ativa de um utilizador.

6.3 Gestão de Planos de Treino

- Registrar Plano de Treino: Permite registar um plano de treino para um utilizador, especificando parâmetros como o número máximo de atividades por dia, a recorrência semanal, o consumo calórico desejado e as atividades a serem incluídas no plano.
- Listar Plano de Treino: Mostra o plano de treino atual de um utilizador.
- Remover Plano de Treino: Permite remover o plano de treino de um utilizador.
- Executar Atividade do Plano de Treino: Permite executar uma atividade específica de um plano de treino, atualizando os detalhes da atividade e registando-a como concluída.
- Verificar Dia de Atividade: Verifica, se num determinado dia, se certa atividade já foi realizada dentro do plano de treino.
- Reiniciar Plano de Atividades: Permite anular o registo de atividades de um plano do utilizador, marcando todas as atividades como não realizadas.



6.4 Estatísticas (*Queries*)

Para fornecer aos utilizadores *insights* valiosos sobre os seus desempenhos e progressos, foram implementadas funcionalidades estatísticas, também conhecidas como "queries". Estas funcionalidades permitem aos utilizadores analisar diferentes aspectos das suas atividades e planos de treino ao longo do tempo.

Todas estas estatísticas têm a opção de introduzir um período temporal para restringir os seus resultados.

6.5 Funcionalidades Adicionais

- Avançar no tempo.
- Guardar o estado da aplicação.

7 Especificações Técnicas das Funcionalidades

A aplicação "GymAtHome" foi desenvolvida com o objetivo de oferecer uma plataforma abrangente para gerir atividades físicas e planos de treino personalizados. Neste capítulo, será detalhado como as principais funcionalidades foram implementadas para proporcionar uma experiência completa aos utilizadores.

7.1 Gestão de Utilizadores

Para facilitar a gestão de utilizadores, foi criada uma classe `UtilizadorManager` que lida com todas as operações relacionadas com utilizadores. Esta implementação permite adicionar novos utilizadores, listar todos os utilizadores registados, atualizar as informações dos utilizadores existentes e remover utilizadores do sistema, através da pesquisa de utilizadores por um número de identificação único (*id*) no mapa de utilizadores. Este mapa serve como base do programa, onde os utilizadores são adicionados, manipulados ou removidos da plataforma.

7.2 Gestão de Atividades Físicas

A gestão de atividades físicas é realizada através de métodos também na classe `UtilizadorManager`. Os utilizadores podem adicionar novas atividades, listar as atividades ativas e terminadas, executar atividades programadas e remover atividades específicas.

Quando um utilizador executa uma atividade, são fornecidos detalhes como a data de início, a data de fim e quaisquer parâmetros específicos da atividade, como distância percorrida, repetições, entre outros. Adicionalmente, o sistema verifica se a atividade pode ser realizada com base em restrições específicas, como a execução de atividades *hard* em dias consecutivos.

7.3 Gestão de Planos de Treino

A gestão de planos de treino é uma funcionalidade essencial do "GymAtHome". Os utilizadores podem registar planos de treino personalizados, definindo parâmetros como o número máximo de atividades por dia, a recorrência semanal e o consumo calórico desejado.

Esta implementação permite adicionar diferentes tipos de atividades a cada dia do plano de treino, garantindo variedade e personalização. Além disso, os utilizadores podem executar atividades específicas dos seus planos de treino, atualizando automaticamente o progresso e registando as atividades e planos de treino concluídos.

7.4 Estatísticas (*Queries*)

- **Atividade Mais Realizada:** Esta funcionalidade identifica a atividade mais frequentemente realizada pelos utilizadores durante um determinado período de tempo. Através da análise das atividades registadas, é determinada a atividade com o maior número de ocorrências.
- **Maior Distância Percorrida:** Esta estatística calcula a distância total percorrida pelos utilizadores em atividades como corrida ou ciclismo. Após a análise das atividades registadas, é determinada a distância acumulada alcançada por um utilizador.
- **Maior Altitude Alcançada:** Similar à funcionalidade de distância, esta estatística calcula a altitude total alcançada pelos utilizadores em atividades que envolvem variações de altitude, como corrida em montanha. Através da análise das atividades registadas, é determinada a soma da altitude alcançada por um utilizador.
- **Utilizador com Maior Número de Atividades:** Esta funcionalidade identifica o utilizador que realizou o maior número de atividades. Analisando as atividades registadas, é determinado o utilizador com o maior número de ocorrências.
- **Utilizador com Mais Calorias Gastas:** Esta estatística calcula o total de calorias gastas pelos utilizadores em todas as atividades realizadas. Através da análise das atividades registadas de cada utilizador e calculada a soma das calorias gastas, é determinado o utilizador que gastou mais calorias.



- **Plano de Treino Mais Exigente:** Esta funcionalidade identifica o plano de treino mais exigente com base no consumo calórico especificado. Analisando os planos de treino registados, é determinado o plano com o maior valor de consumo calórico, indicando assim o plano de treino mais desafiador.

7.5 Funcionalidades Adicionais

Além das funcionalidades principais mencionadas acima, "GymAtHome" também oferece algumas funcionalidades adicionais, como a capacidade de guardar as informações registadas a qualquer momento, garantindo que os utilizadores possam manter um registo completo das suas atividades e progresso ao longo do tempo.

Outra funcionalidade importante é a opção de "salto no tempo", que permite aos utilizadores observar as restrições temporais quanto à execução de atividades, como mencionado anteriormente na execução de atividades *Hard*.

8 Conclusão

Ao concluirmos este projeto prático da Unidade Curricular Programação Orientada aos Objetos (POO), podemos afirmar que foram utilizadas metodologias e abordagens adequadas para resolver o problema proposto.

Com o desenvolvimento completo das funcionalidades solicitadas pelos Docentes, consideramos que o trabalho foi bem executado e atingiu todos os objetivos menos o último ponto do enunciado referente à automatização da criação de planos de treino. Durante o processo de desenvolvimento do sistema/aplicação, foi necessário aplicar conceitos teóricos de *POO*, o que possibilitou um melhor entendimento da linguagem utilizada (*Java*) e da forma como as classes e objetos podem ser implementados de maneira eficiente.

A adoção de um modelo de boas práticas de desenvolvimento de sistemas de software (**MVC**) permitiu a entrega de um produto final que nos parece adequado face aos requisitos desta UC.